

Aprendizaje Automático (Machine Learning)

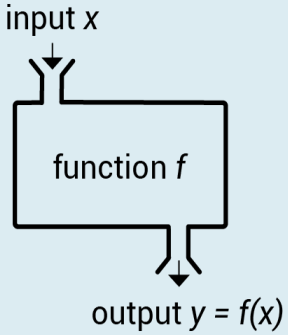
Clase 32: Árboles de decisión, Bagging & Bosque de árboles

Pedro Ramos

Recordemos

¿Qué son los Algoritmos Supervisados?

En los algoritmos de Aprendizaje Supervisado la data es de la forma $\{x_i, y_i\}_{i=1}^n$ donde $X = \{x_i\}_{i=1}^n$ se denominan *inputs* o *variables de entrada* e y_i es el *output* o *variable de salida*. El modelo representa la *función de mapeo* $f(X)$ de los inputs a los outputs:



Dependiendo de la naturaleza de la variable respuesta Y, el modelo es de clasificación (si Y es categórica) o de regresión (si Y es numérica). Es importante reconocer la naturaleza de la variable respuesta pues a partir de esta, descartamos los modelos que no serían adecuados para el caso de estudio.

Árbol de decisión

Árbol de decisión

Corresponde a una técnica de Machine Learning de tipo Supervisado. Al final del entrenamiento, se obtienen secuencias de preguntas de los datos que llevan a un resultado o *predicción*. Un plus que tiene esta técnica es que es posible visualizar las secuencias de preguntas en forma de árbol, siendo fácil de explicar y comprender para la audiencia.

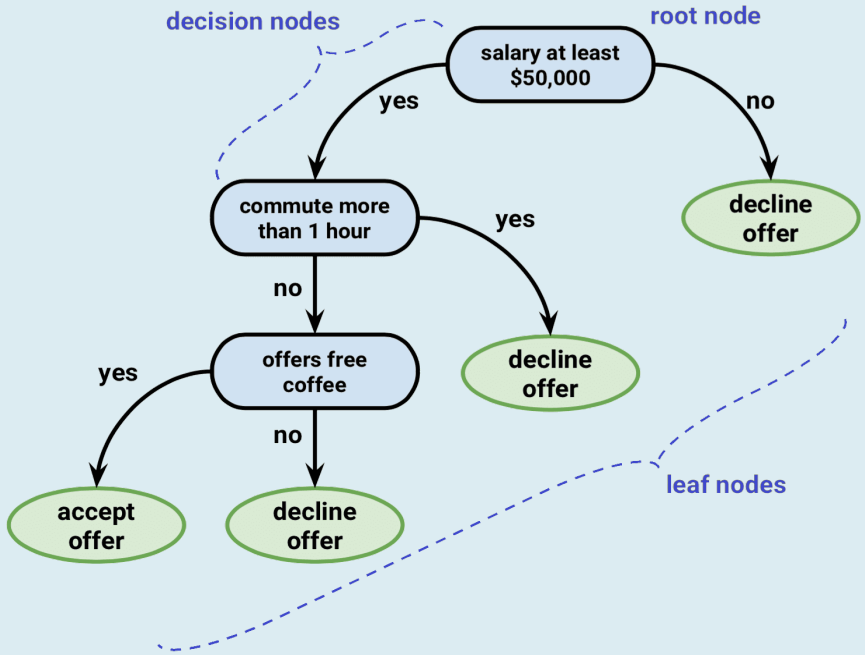
Los árboles de decisión se clasifican en:

- **Árbol de Regresión:** Cuando la variable Y output es numérica. Ejemplo: Determinar las secuencias de preguntas adecuadas de modo de predecir el salario de un colaborador.
- **Árbol de Clasificación:** Cuando la variable Y output es categórica. Ejemplo: Determinar las secuencias de preguntas adecuadas de modo de predecir el fallo o no fallo de un producto.

Ejemplo Árbol de Clasificación: ¿Debería aceptar un nuevo empleo?

Podemos observar las siguientes partes del árbol:

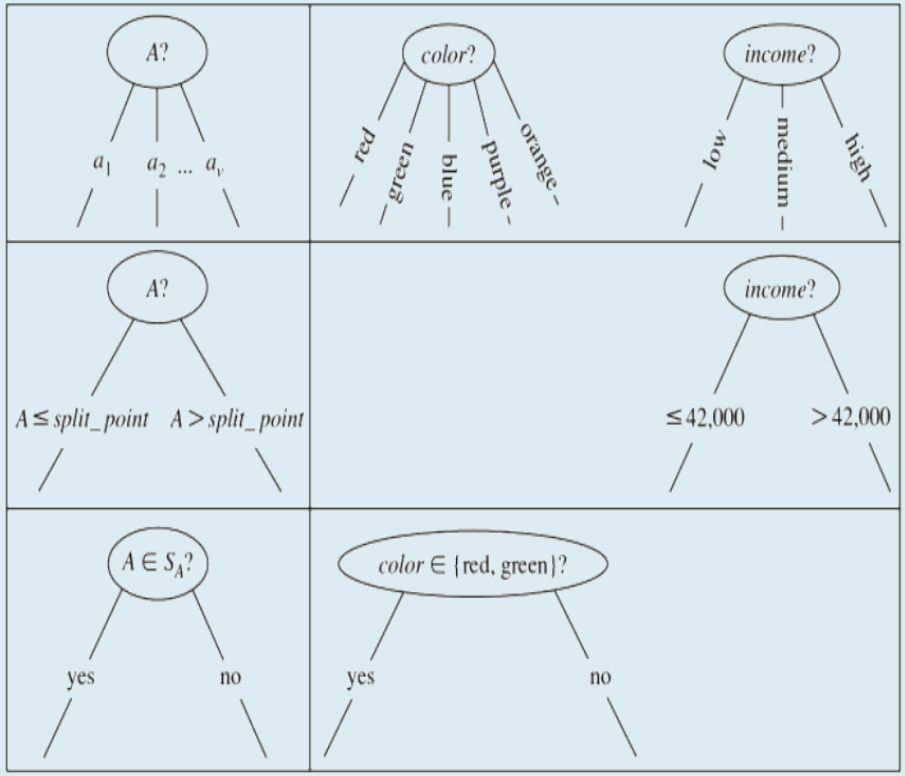
- Cada nodo de decisión (decision node) representa una pregunta sobre un atributo. El nodo raíz (root node) corresponde a la primera pregunta.
- Cada rama representa una posible respuesta a las preguntas.
- Cada hoja o nodo terminal (leaf node) representa la clase o predicción al seguir ese camino.



Árbol de clasificación: ¿Cómo se obtienen las preguntas?

1. Si todas las observaciones pertenecen a la misma clase o categoría, se crea un nodo terminal con dicha categoría.
2. Si no, se recurre a algún método para determinar la primera pregunta o criterio de separación, la idea es obtener particiones puras (que los grupos estén en la menor medida posible mezclados, es decir más homogéneos).
3. Se crea el primer nodo el criterio de separación encontrado, este proceso se itera de forma de añadir ramas y nodos de forma anidada hasta que ocurra alguno de los siguientes casos:

- Todas las tuplas pertenecen a la misma clase.
- No hay más atributos para particionar.
- Ya no hay más datos.



Criterios de ramificación

Hasta ahora comprendemos el proceso de crecimiento del árbol, pero necesitamos saber cómo discernir entre usar una u otra variable para crear los nodos de decisión.

Algunos métodos o criterios de ramificación que existen son:

- Entropía o Ganancia de información
- Impureza Gini

Entropía o Ganancia de información

Este método trata de representar cuánta información he ganado particionando por cierto atributo y se relaciona con el grado de impureza del nodo.

- Si el resultado al separar por un atributo es obvio (gran impureza del nodo 50%-50%), no he ganado información, por lo tanto, no vale mucho la pena utilizar ese atributo.
- Si el resultado al separar por un atributo no era tan obvio (baja impureza 90%-10%), he ganado mucha información, porque no era algo obvio o esperable.

Entropía o Ganancia de información

La información esperada necesaria o entropía, para clasificar una tupla en alguna de las $C_1, \ldots, C_m$ clases es:

$$\text{Info}(D) = -\sum_{i=1}^{m} p_i log_2(p_i)$$

Donde p_i consiste en la probabilidad de pertenecer a la clase C_i. Para saber cuánto es el grado de entropía luego de la partición sugerida por el atributo A, calculamos:

$$\text{Info}_A(D) = \sum_{j=1}^{v} q_j \cdot \text{Info}(D_j)$$

Donde q_j corresponde a la probabilidad de pertenecer a la categoría j del atributo A.

Ejemplo: ¿La cancha se encuentra habilitada para jugar?

La siguiente tabla contiene datos de distintos días donde la variable Tenis indica si la cancha estaba o no habilitada para jugar. A modo de ejemplo calcularemos la entropía para el atributo Pronóstico.

Las probabilidades globales de pertenecer a cada clase C_i son:

$$p_{si} = \frac{8}{14} \text{ y } p_{no} = \frac{6}{14}$$

La entropía global $\text{Info}(D)$ es:

$$-\frac{8}{14}log_2(\frac{8}{14}) - \frac{6}{14}log_2(\frac{6}{14}) = 0.985$$

Día	Temperatura	Pronóstico	Humedad	Viento	Tenis
5-Jul	caluroso	soleado	alta	no	no
6-Jul	caluroso	soleado	alta	si	no
7-Jul	caluroso	nublado	alta	no	si
9-Jul	frio	lluvia	normal	no	si
10-Jul	frio	nublado	normal	si	si
12-Jul	templado	soleado	alta	no	no
14-Jul	frio	soleado	normal	no	si
15-Jul	templado	lluvia	normal	no	si
20-Jul	templado	soleado	normal	si	si
21-Jul	templado	nublado	alta	si	no
22-Jul	caluroso	nublado	normal	no	si
23-Jul	templado	lluvia	alta	si	no
26-Jul	frio	lluvia	normal	si	no
30-Jul	templado	lluvia	alta	no	si

Ejemplo: ¿La cancha se encuentra habilitada para jugar?

Para calcular la entropía de pronóstico, debemos calcular las probabilidades de cada categoría en pronóstico:

$q_{soleado} = \frac{5}{14}$, $q_{nublado} = \frac{4}{14}$ y $q_{lluvia} = \frac{5}{14}$

Y las probabilidades de cada clase en cada categoría del atributo pronóstico son:

$p_{si,soleado} = \frac{2}{5}$ y $p_{no,soleado} = \frac{3}{5}$
$p_{si,nublado} = \frac{3}{4}$ y $p_{no,nublado} = \frac{1}{4}$
$p_{si,lluvia} = \frac{3}{5}$ y $p_{no,lluvia} = \frac{2}{5}$

Día	Temperatura	Pronóstico	Humedad	Viento	Tenis
5-Jul	caluroso	soleado	alta	no	no
6-Jul	caluroso	soleado	alta	si	no
7-Jul	caluroso	nublado	alta	no	si
9-Jul	frio	lluvia	normal	no	si
10-Jul	frio	nublado	normal	si	si
12-Jul	templado	soleado	alta	no	no
14-Jul	frio	soleado	normal	no	si
15-Jul	templado	lluvia	normal	no	si
20-Jul	templado	soleado	normal	si	si
21-Jul	templado	nublado	alta	si	no
22-Jul	caluroso	nublado	normal	no	si
23-Jul	templado	lluvia	alta	si	no
26-Jul	frio	lluvia	normal	si	no
30-Jul	templado	lluvia	alta	no	si

Y la entropía asociada a la variable pronóstico $\text{Info}_{pronóstico}(D)$ es:

$$\frac{5}{14}[-\frac{2}{5}log_2(\frac{2}{5}) - \frac{3}{5}log_2(\frac{3}{5})] + \frac{4}{14}[-\frac{3}{4}log_2(\frac{3}{4}) - \frac{1}{4}log_2(\frac{1}{4})] + \frac{5}{14}[-\frac{3}{5}log_2(\frac{3}{5}) - \frac{2}{5}log_2(\frac{2}{5})] = 0.925$$

Ejemplo: ¿La cancha se encuentra habilitada para jugar?

Si calculamos la entropía para los atributos Humedad y Viento de manera análoga, obtenemos:

Atributo	Entropía
Pronóstico	0.925
Humedad	0.833
Viento	0.936

¿Cuál de estos atributos elegiríamos para el nodo raíz? Según el método de entropía, aquél que tenga la menor entropía o impureza, en este caso sería Humedad (menor impureza mayor ganancia de información), por lo tanto la primera pregunta sería *¿La humedad es alta?*

Para seguir con el árbol se calculan las entropías dado que ya está separado por humedad y así sucesivamente hasta llegar a algún criterio de parada.

Impureza de Gini

Representa la probabilidad de que una muestra seleccionada al azar de un nodo se clasifique incorrectamente de acuerdo con la distribución de muestras en el nodo.

Al entrenar un árbol de decisión, la mejor división se elige minimizando la impureza de Gini, que se calcula restando las impurezas ponderadas de las ramas de la impureza original.

Impureza de Gini

Los datos se dividen según el atributo A, en dos regiones, A_1 y A_2, con n_1 y n_2 elementos, respectivamente. p_{ji} corresponde a la proporción de elementos en A_j con $j = 1, 2$ que pertenecen a la clase i con $i = 1, \ldots, m$.

La impureza de Gini $I(A_j)$ se calcula:

$$= p_{j1}(1 - p_{j1}) + p_{j2}(1 - p_{j2}) + \ldots + p_{jm}(1 - p_{jm})$$

$$= \sum_{i=1}^{m} p_{ji}(1 - p_{ji}) = \sum_{i=1}^{m} (p_{ji} - p_{ji}^2) = \sum_{i=1}^{m} p_{ji} - \sum_{i=1}^{m} p_{ji}^2$$

$$= 1 - \sum_{k=1}^{m} p_{ji}^2$$

Y el índice de Gini se calcula:

$$\text{Gini}(A) = q_1 I(A_1) + q_2 I(A_2)$$

Donde q_j corresponde a la probabilidad de pertenecer a la categoría j del atributo A.

Ejemplo: Retorno de préstamos

En el siguiente ejemplo, se tienen variables de distintos préstamos e interesa predecir la variable Return.

Para calcular el índice de Gini para el atributo Past Trend, debemos calcular las probabilidades de cada categoría del atributo:

$$q_{Positive} = \frac{6}{10} \text{ y } q_{Negative} = \frac{4}{10}$$

Y las probabilidades de cada clase en cada categoría del atributo Past Trend son:

$$p_{up,Positive} = \frac{4}{6} \text{ y } p_{down,Positive} = \frac{2}{6}$$
$$p_{up,Negative} = \frac{0}{4} \text{ y } p_{down,Negative} = \frac{4}{4}$$

Past Trend	Open Interest	Trading Volume	Return
Positive	Low	High	Up
Negative	High	Low	Down
Positive	Low	High	Up
Positive	High	High	Up
Negative	Low	High	Down
Positive	Low	Low	Down
Negative	High	High	Down
Negative	Low	High	Down
Positive	Low	Low	Down
Positive	High	High	Up

Ejemplo: Retorno de préstamos

Luego, las impurezas se pueden obtener como siguen:

$$I(A_{Positive}) = 1 - ((4/6)^2 + (2/6)^2) = 0.44$$

$$I(A_{Negative}) = 1 - ((0)^2 + (4/4)^2) = 0$$

Y el índice de Gini del atributo Past Trend es:

$$\text{Gini}(PastTrend) = \frac{6}{10} \cdot 0.44 + \frac{4}{10} \cdot 0 = 0.27$$

Ejemplo: Retorno de préstamos

Si realizamos el mismo cálculo para los atributos Open Interest y Trading Volume obtenemos:

Atributo	Índice de Gini
Past Trend	0.27
Open Interest	0.47
Trading Volume	0.34

En base al criterio de Gini, se elegiría como nodo raíz el atributo Past Trend. Por lo tanto, la primera pregunta sería *¿Past Trend es Positivo?*.

Para seguir con el árbol se calculan los índices de Gini dado que ya está separado por Past Trend y así sucesivamente hasta llegar a algún criterio de parada.

Algunas aplicaciones de árboles de decisión

- Física: detección de partículas
- Ingeniería biomédica: identificar dispositivos implantables
- Análisis financiero: satisfacción del cliente
- Astronomía: clasificación de estrellas
- Medicina: diagnóstico

Entre muchas otras.

Ejemplo en R

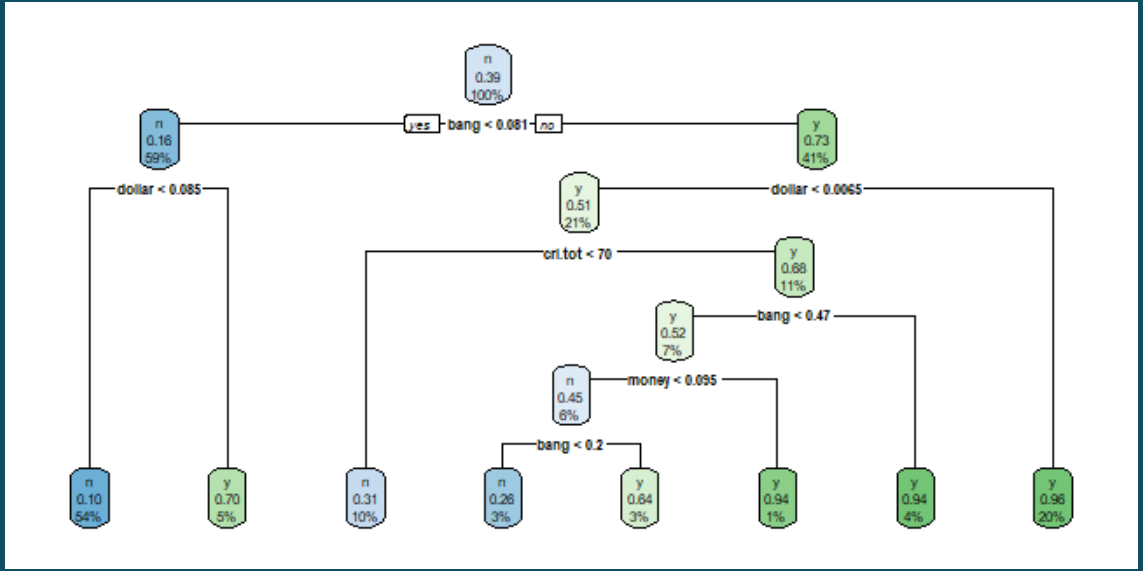
La base de datos spam7 de la librería DAAG contiene 6 posibles variables explicativas y la variable target es *yesno* que corresponde a si el correo es o no spam.

```
library(DAAG)
library(splitTools)
set.seed(123)

ind_train <- partition(1:nrow(spam7), p=c(0.7,0.3))
train <- spam7[ind_train$`1`,]
test <- spam7[ind_train$`2`,]
```

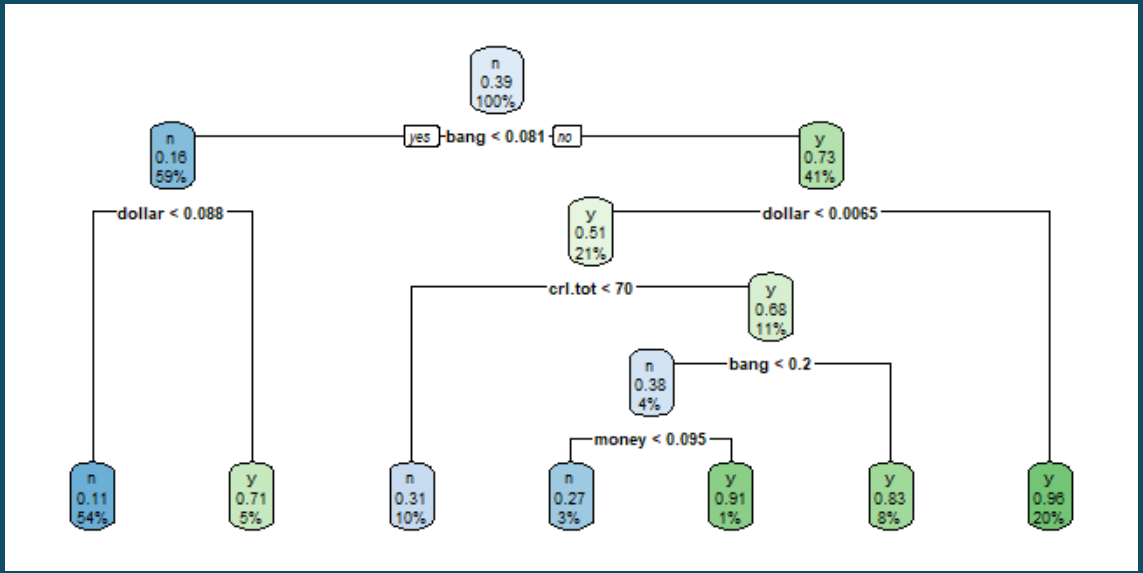
Árbol de decisión con método de Entropía

```
library(rpart); library(rpart.plot)
tree_entropy <- rpart(yesno ~., data = train,
                      parms = list(split = 'information'),
                      method = "class")
rpart.plot(tree_entropy)
```



Árbol de decisión con método de Gini

```
tree_gini<- rpart(yesno~., data = train,
                  parms = list(split='gini'), method = "class")
rpart.plot(tree_gini)
```



Ejemplo en Python

La base de datos wine son los resultados de un análisis químico a tres tipos de vinos, contiene 13 variables numéricas además de la variable target.

```
from sklearn import datasets
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree

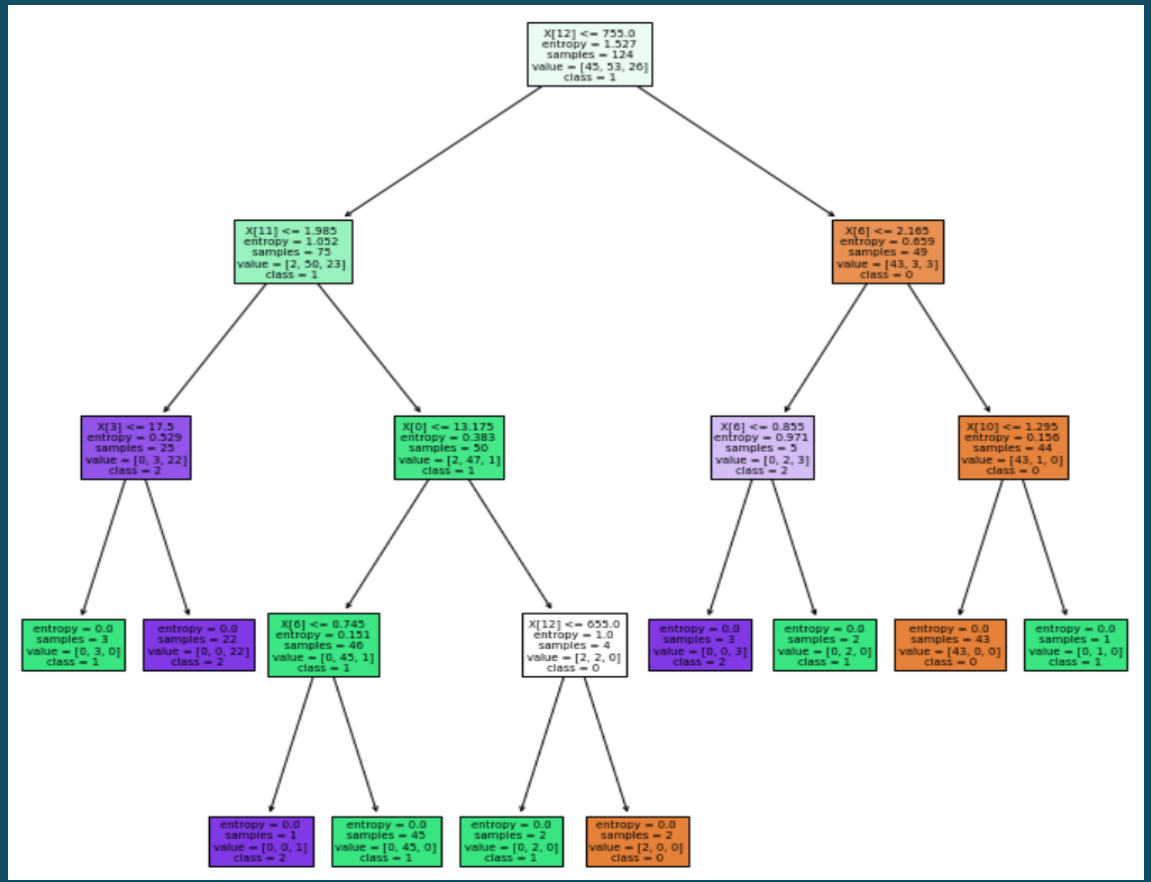
X, y = datasets.load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=123)
```

Árbol de decisión con método de Entropía

```
tree_entropy = DecisionTreeClassifier(criterion='entropy').fit(
  X_train, y_train)

fig, ax = plt.subplots(figsize=(12, 12))
plot_tree(tree_entropy, filled=True, class_names = ['0', '1', '2'])
```

Árbol de decisión con método de Entropía

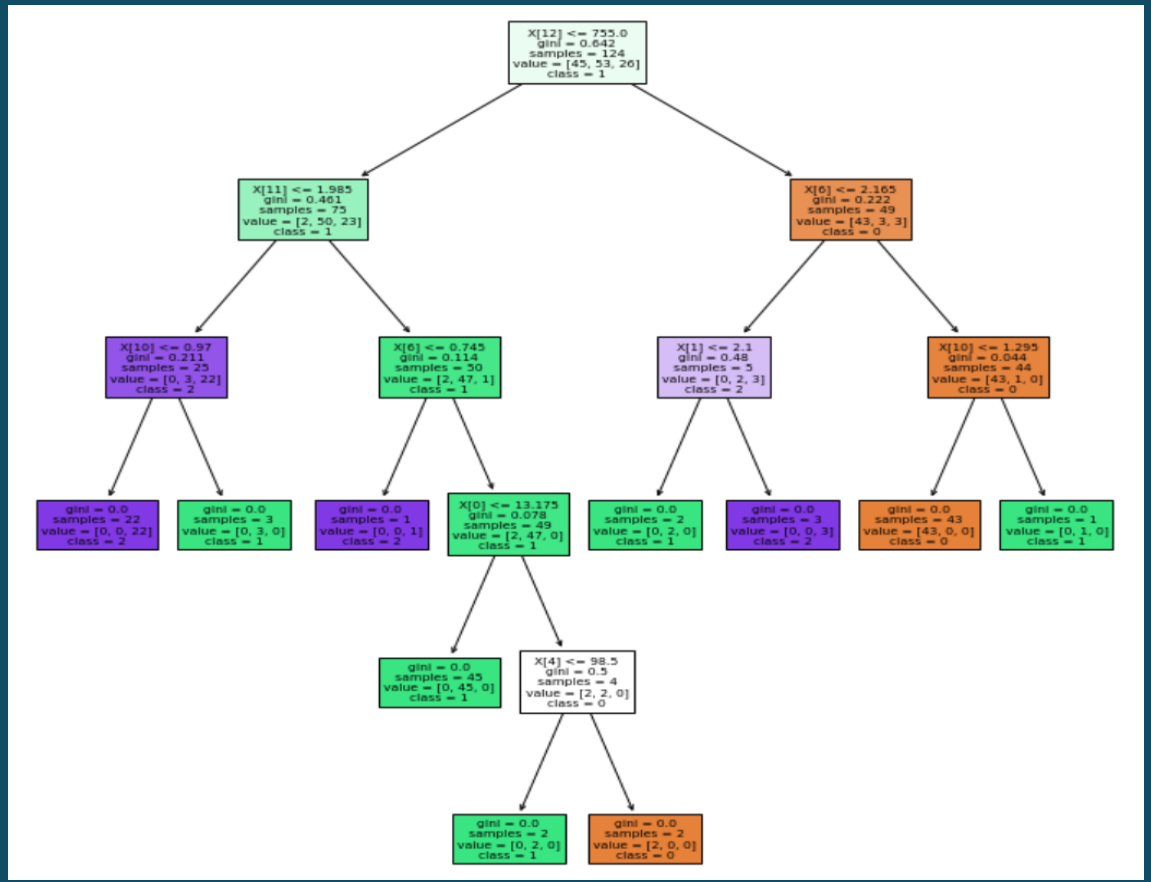


Árbol de decisión con método de Gini

```
tree_gini = DecisionTreeClassifier(criterion='gini').fit(X_train, y_train)

fig, ax = plt.subplots(figsize=(12, 12))
plot_tree(tree_gini, filled=True, class_names = ['0', '1', '2'])
```

Árbol de decisión con método de Gini



Ventajas y Desventajas de los Árboles de decisión

Ventajas	Desventajas
Construcción barata	Fácil de sobreajustar
Rápido en clasificar registros nuevos	Muy sensible ante cambios del set de entrenamiento
Fácil de interpretar (árboles pequeños)	Difícil de interpretar (árboles grandes)
Excluye características sin importancia	Los modelos pueden inclinarse hacia divisiones de atributos con gran cantidad de niveles

Algunas variantes que pueden probarse al utilizar árboles de decisión:

- Podar el árbol
- Bagging
- Definir profundidad máxima, cantidad de features, entre otros
- Bosques de árboles

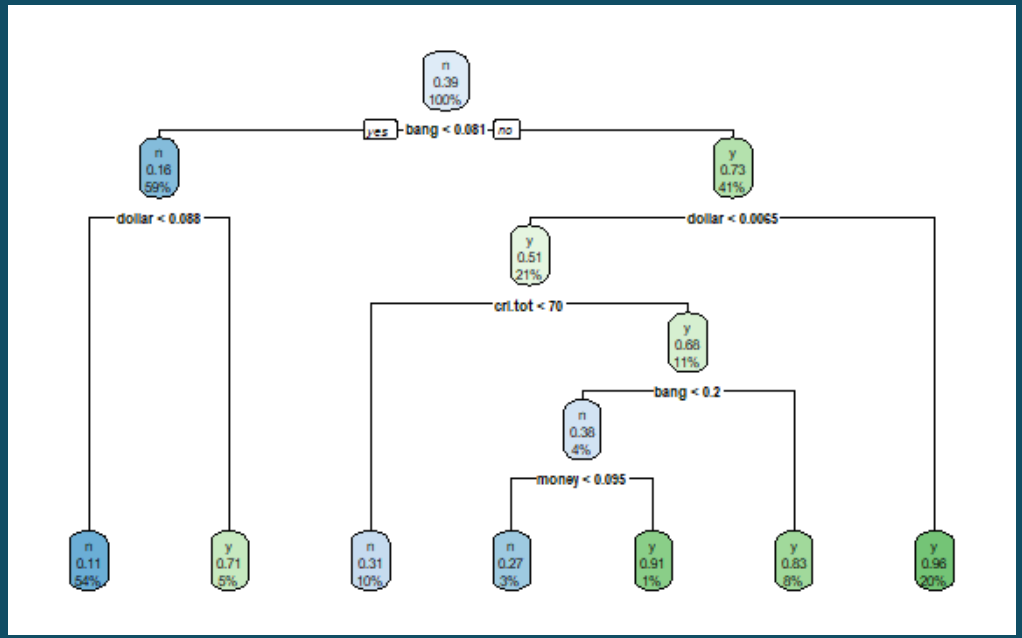
Podar el árbol

En ocasiones, el árbol obtenido en una primera iteración es excesivamente grande y profundo y esto puede indicar peligro de sobreajuste de nuestro modelo.

La idea es sencilla, a medida que el árbol crece más, se vuelve más complejo y cualquier división adicional que no aporta significativamente, debe ser eliminado. En estos casos podemos *podar* el árbol utilizando el parámetro de complejidad.

Ejemplo en R

Recordemos nuestro árbol anterior construido con el método de Gini:



¿Cada ramificación realmente aporta valor al modelo?

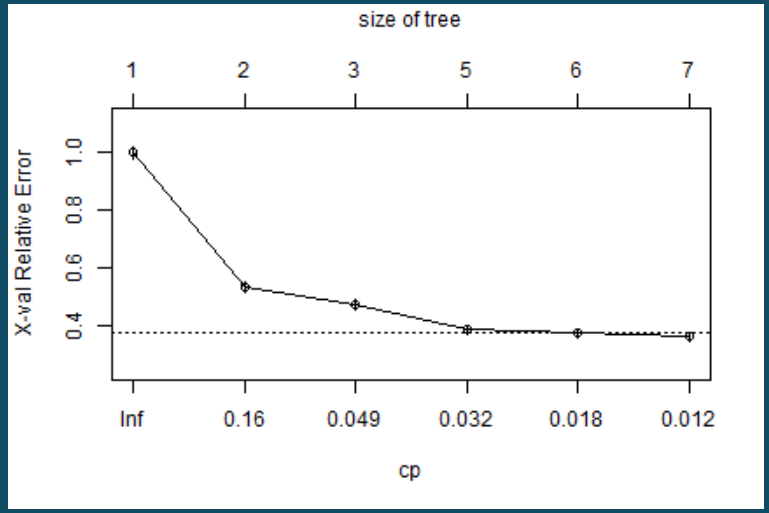
Costo de complejidad en R

```
printcp(tree_gini)
```

```
##
## Classification tree:
## rpart(formula = yesno ~ ., data = train, method = "class", parms = list(split =
##
## Variables actually used in tree construction:
## [1] bang    crl.tot dollar  money
##
## Root node error: 1270/3220 = 0.39441
##
## n= 3220
##
##         CP nsplit rel error  xerror     xstd
## 1 0.485827      0   1.00000 1.00000 0.021837
## 2 0.052756      1   0.51417 0.53543 0.018236
## 3 0.046063      2   0.46142 0.47244 0.017398
## 4 0.022835      4   0.36929 0.38661 0.016062
## 5 0.014173      5   0.34646 0.37480 0.015859
```

Costo de complejidad en R

```
plotcp(tree_gini)
```



La idea se trata de determinar desde qué tamaño del árbol el error relativo ya no cambia significativamente. En este caso, podríamos considerar que cuando el cp=0.032 ya no se observa una reducción drástica en el error relativo.

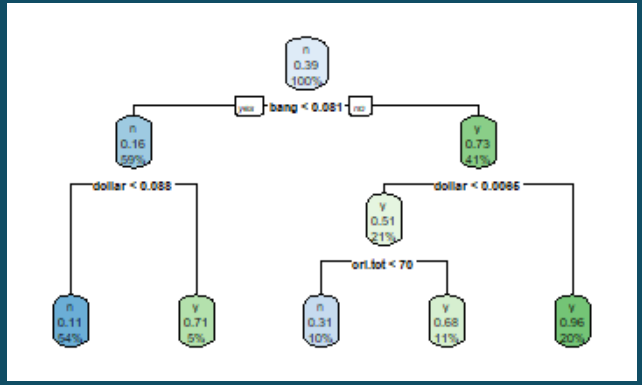
Poda de un árbol por costo de complejidad en R

Definimos el costo de complejidad como parámetro de control del árbol:

```
tree_gini_prune<- rpart(yesno~., data=train, parms=list(split='gini'),
                        method="class", control=rpart.control(cp=0.032))
```

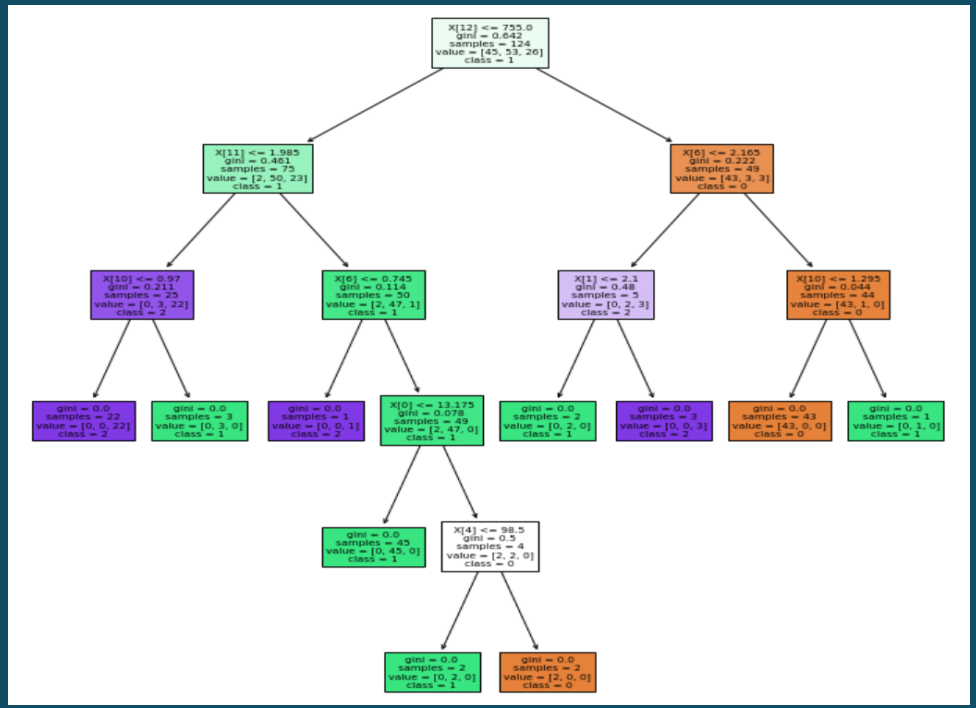
Y observamos el nuevo árbol podado:

```
rpart.plot(tree_gini_prune)
```



Ejemplo en Python

Recordemos nuestro árbol anterior construido con el método de Gini:



¿Cada ramificación realmente aporta valor al modelo?

Costo de complejidad en Python

Con el objetivo de identificar la profundidad óptima que consigue reducir la varianza y aumentar la capacidad predictiva del modelo, se somete al árbol a un proceso de pruning. Esto se puede medir directamente con la complejidad o con el número máximo de profundidad del árbol.

Con validación cruzada, se puede probar diferentes valores de estos dos hiperpárametros en una grilla, y se eligen los valores que tengan mejor desempeño en el proceso de entrenamiento.

```
param_grid = {'ccp_alpha':np.linspace(0, 0.5, 10),
              'max_depth': np.arange(2, 7)}

from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(
        estimator = DecisionTreeClassifier(
            max_depth = None, random_state = 123, criterion = 'gini'),
        param_grid = param_grid,
        scoring    = 'accuracy',
        cv         = 10,
        refit      = True,
        return_train_score = True)
```

Costo de complejidad en Python

La función logra identificar los hiperpárametros que lograron mejor desempeño en el scoring señalado, es decir, tuvieron mejor desempeño según el acurracy.

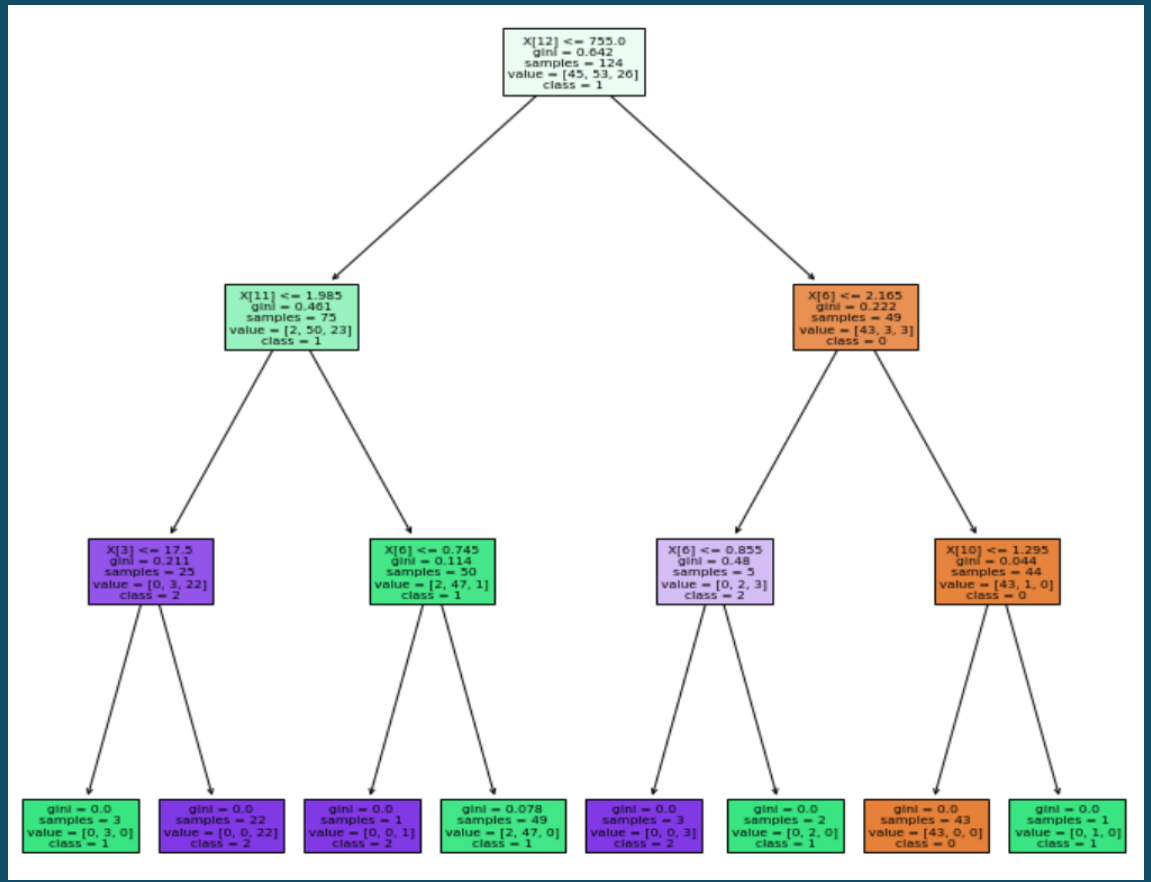
```
grid.fit(X_train, y_train)
grid.best_params_
```

{'ccp_alpha': 0.0, 'max_depth': 3}

Y observamos el nuevo árbol podado:

```
modelo_final = grid.best_estimator_
fig, ax = plt.subplots(figsize=(12, 12))
plot_tree(modelo_final, filled=True, class_names = ['0', '1', '2'])
```

Costo de complejidad en Python



Bagging

Bagging

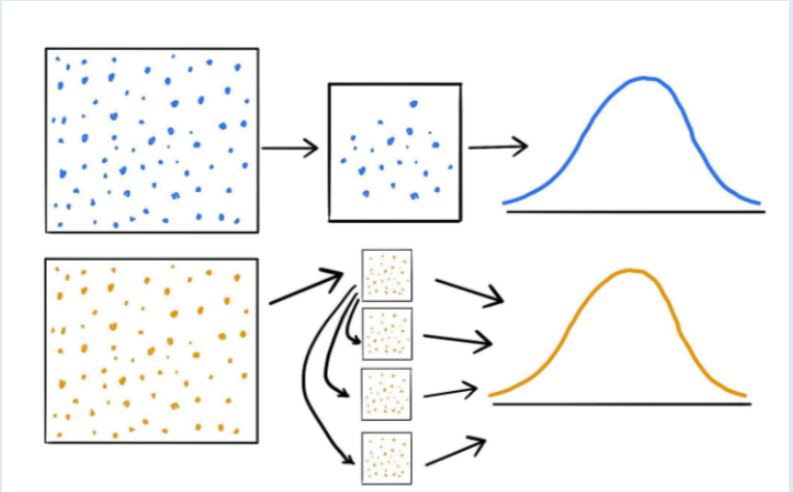
Bootstrap

El Bootstrap es un método de remuestreo que consiste en calcular un estadístico, tomando muestras con reemplazo una cantidad de veces igual al tamaño de la data y replicando el proceso múltiples veces. Las distintas realizaciones del estadístico es lo que se conoce como la muestra Bootstrap y esta simula la distribución del estadístico.

Para una muestra de tamaño n

- Muestrear n veces con reemplazo
- Calcular T(x), el estadístico objetivo para la muestra tomada en el paso 1.
- Repetir los pasos B veces.

Lo que se obtendrá son B realizaciones del estadístico T(x).



Bagging

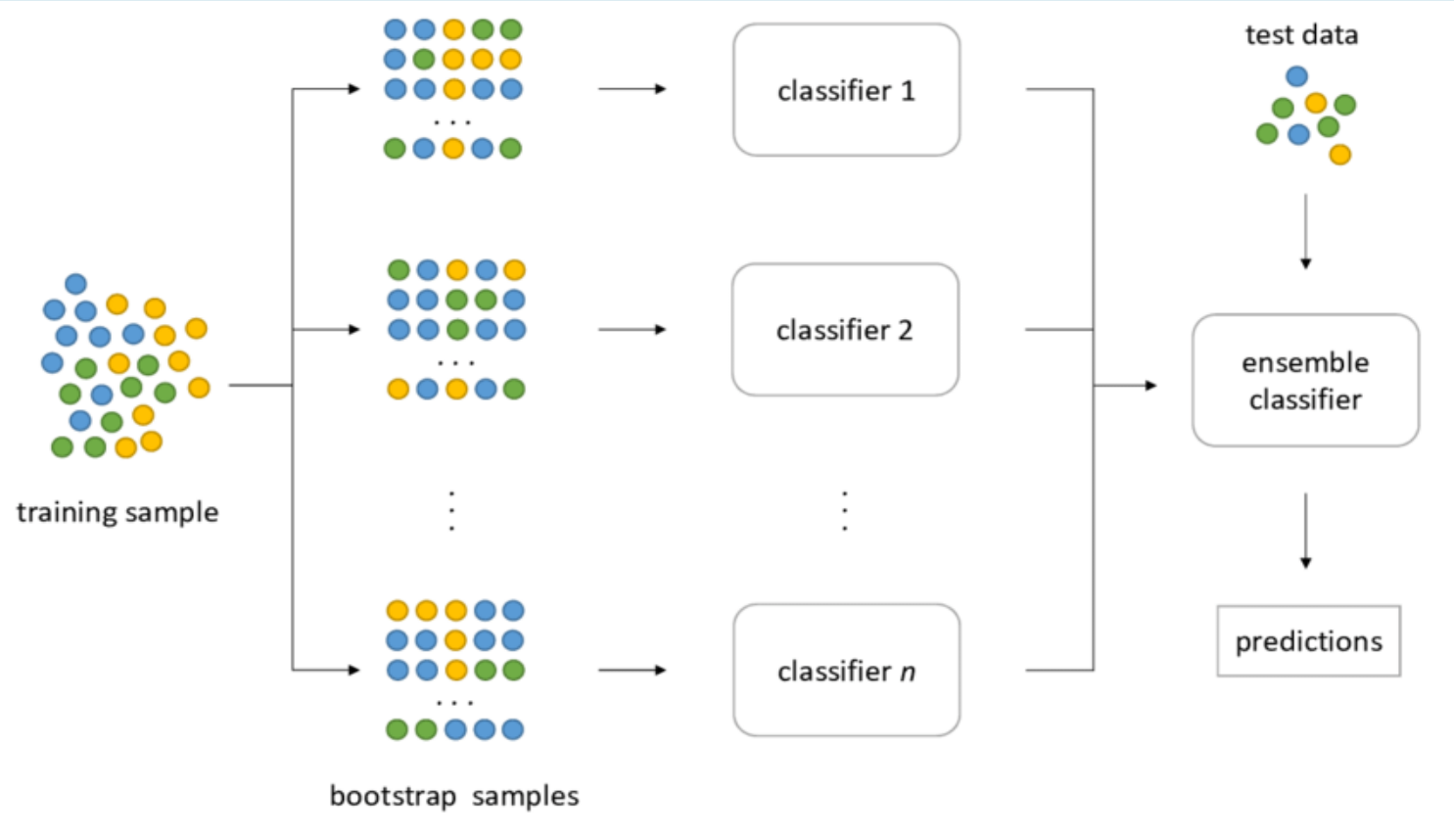
Bootstrap aggregation

Dado que esto se puede aplicar a un estadístico, ¿Que tan alocada es la idea de aplicarla a un modelo supervisado?

Esto es lo que se conoce como método de bagging o *bootstrap aggregation*, es un método de ensamblaje que ejecuta muchos modelos con remuestreos a los datos, para luego agregar sus predicciones con el fin de obtener una predicción más robusta.

En efecto, se hacen B muestras, se ajusta un modelo a cada una de ellas y luego se les hace competir a los modelos, esto es el paso de agregación. Cuando el modelo es de regresión, esta suele ser el promedio de las predicciones de los modelos, mientras que en un clasificador es la clase mayoritaria.

Bagging


test data
classifier 1
classifier 2
classifier n
ensemble classifier
predictions
training sample
bootstrap samples

Bagging

Los árboles de decisión sufren de alta varianza, lo que significa que, si dividiéramos al azar los datos de entrenamiento en dos grupos y ajustáramos un árbol de decisión a cada mitad, los resultados que obtendríamos podrían ser radicalmente diferentes.

El método de bagging es una técnica que se utiliza para reducir la varianza de un método de aprendizaje estadístico, usado muy frecuentemente con árboles de decisión.

Procedimiento

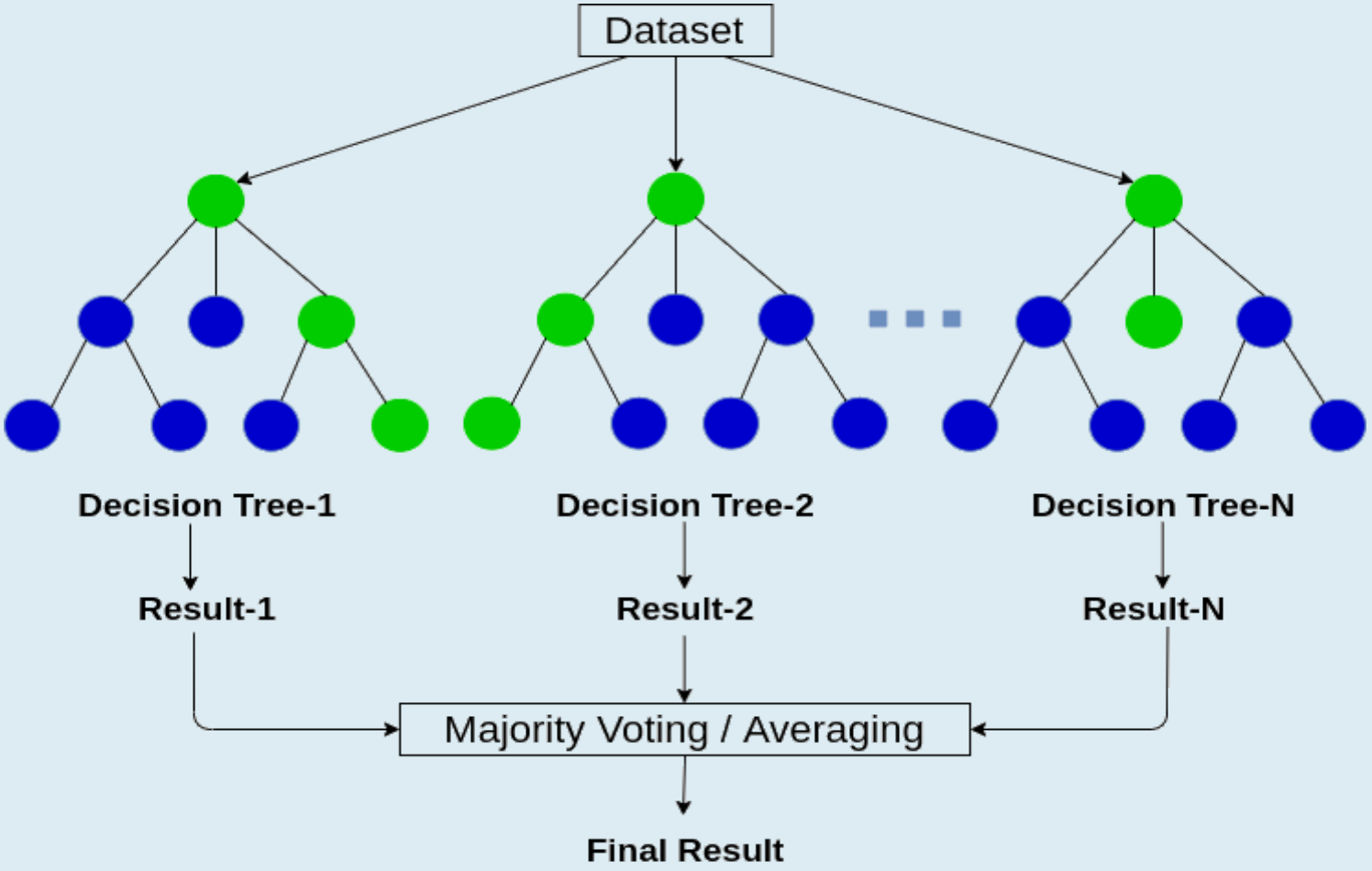
1. Producir varios conjuntos de entrenamiento.
2. Crear para cada conjunto de entrenamiento, un modelo (en este caso, un árbol de decisión).
3. Combinar las predicciones de todos estos modelos obtenidos.

Bosque de árboles

Bosque de árboles

Corresponde a una técnica de Machine Learning que se basa en un conjunto de árboles de decisiones seleccionando aleatoriamente submuestras (con reemplazamiento) para elaborar cada árbol.

El punto clave del Bosque de árboles es utilizar una serie de árboles de decisión (diferentes individuos y diferentes variables), con el fin de mejorar la tasa de clasificación correcta. La diferencia con el bagging es que en el bosque de árboles también se toma una muestra de los features, es decir, no se utilizan todas las variables como en el bagging.


Dataset
Decision Tree-1
Decision Tree-2
Decision Tree-N
Result-1
Result-2
Result-N
Majority Voting / Averaging
Final Result

Importancia

Si bien es cierto que el proceso de Random Forest consigue mejorar la capacidad predictiva en comparación a los modelos basados en un único árbol, esto tiene un coste asociado, la interpretabilidad del modelo se reduce. Al tratarse de una combinación de múltiples árboles, no es posible obtener una representación gráfica sencilla del modelo y no es inmediato identificar de forma visual qué predictores son más importantes.

Sin embargo, se han desarrollado nuevas estrategias para cuantificar la importancia de los predictores. Dos de estas medidas son:

- **Importancia por permutación:** Identifica la influencia que tiene cada predictor sobre una determinada métrica de evaluación del modelo (mse, classification error). Si el predictor permutado estaba contribuyendo al modelo, es de esperar que el modelo aumente su error, ya que se pierde la información que proporcionaba esa variable.

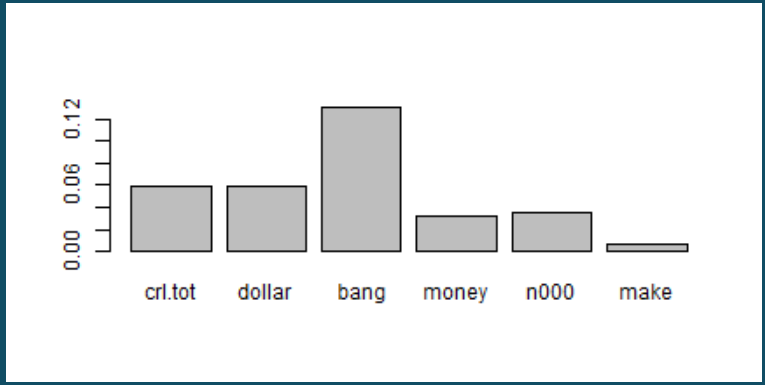
- **Impureza de nodos:** Cuantifica el incremento total en la pureza de los nodos debido a divisiones en las que participa el predictor (promedio de todos los árboles).

Ejemplo en R

```
library(randomForest)

set.seed(2022)
RanForClas <- randomForest(yesno ~., data = train,
            ntree = 100, importance = T)

barplot(RanForClas$importance[,'MeanDecreaseAccuracy'],
names.arg=names(RanForClas$importance[,'MeanDecreaseAccuracy']))
```



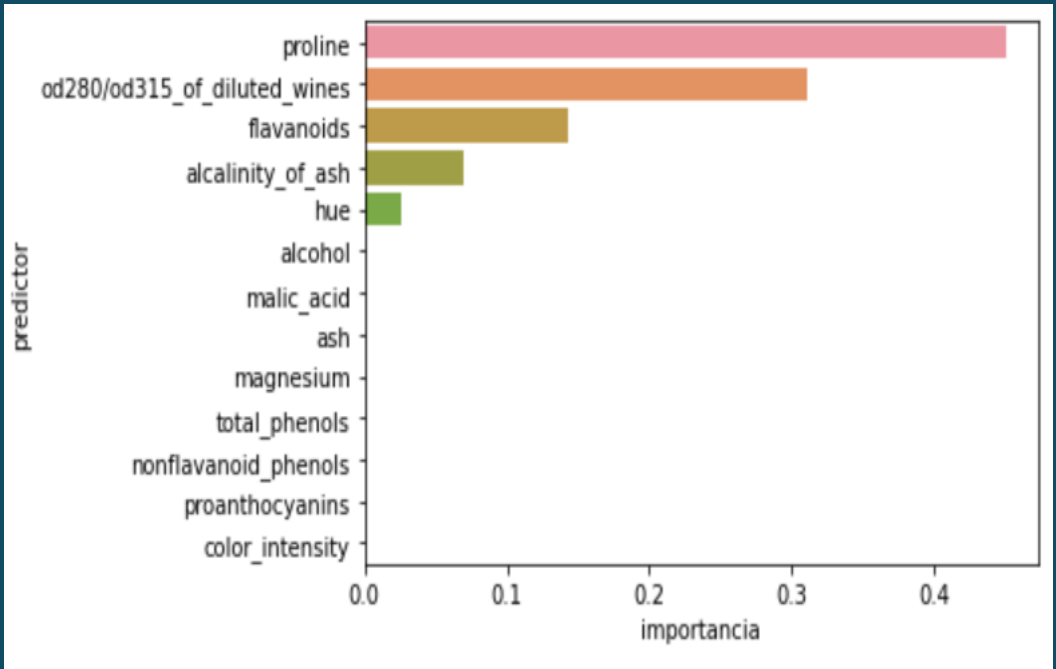
Ejemplo en Python

```
from sklearn.ensemble import RandomForestClassifier
RanForClas = RandomForestClassifier(random_state = 2022,
  n_estimators = 100).fit(X_train, y_train)

importancia_pred = pd.DataFrame(
    {'predictor': datasets.load_wine().feature_names,
    'importancia': modelo_final.feature_importances_}
  ).sort_values('importancia', ascending=False
  ).reset_index(drop = True)
```

Ejemplo en Python

```
import seaborn as sns
sns.barplot(x="importancia", y="predictor", data=importancia_pred)
```



Ventajas y Desventajas de los bosques de árboles

Ventajas	Desventajas
Puede manejar miles de variables de entrada y reconocer las significativas	Pérdida de interpretación
Una de las salidas del modelo es el feature importance	Bueno para clasificación, no tanto para regresión
Es posible usarlo en otros casos (clustering y detección de anomalías)	Poco control en lo que hace (caja negra)

¡Gracias!